

Yapay Zekâya Mimari Tanımlama: Geometri-Önce Yaklaşımıyla Sistem Tasarımı

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Modern yapay zekâ sistemleri kod üretme konusunda güçlüdür; ancak doğru yönlendirilmediklerinde çoğu zaman yalnızca yerel problemleri çözen, kısa vadeli ve yüksek teknik borç üreten yapılar ortaya çıkarırlar. Bu nedenle yapay zekâya yalnızca “kod yaz” demek yeterli değildir. Asıl belirleyici olan, sistemin geometrisini tanımlamak ve modelin akıl yürütebileceği yapısal sınırları oluşturmaktır.

Bu çalışma, yapay zekâ destekli sistem geliştirmede “geometri-önce” (geometry-first) olarak adlandırılabilir bir yaklaşımı sistematikleştirmektedir. Yaklaşımın özü şudur: kod üretiminden önce sistemin amacı, alan modeli, sınırları, durum organizasyonu, veri akış yönleri, hata modeli, ölçek varsayımları, kod organizasyonu, test yapısı ve mimari prensipleri açık biçimde tanımlanır; model bu tanımlı uzay içinde üretim yapar.

Çalışma boyunca örnekler, jenerik bir gerçek zamanlı iş birliği platformu üzerinden verilmektedir: çok katılımcılı oturumları, gerçek zamanlı veri akışını, yapay zekâ destekli özetlemeyi ve kayıt yönetimini destekleyen bir sistem. Örneklerin amacı belirli bir teknolojiyi önermek değil, tanımlama pratiğinin biçimini göstermektir.

Sistem Amacının Tanımlanması

Mimari üretimin ilk adımı, sistemin neden var olduğunun net biçimde tanımlanmasıdır. Örneğin:

`A collaboration platform that supports realtime transcription,
AI-assisted summaries, session recording,
and multi-participant sessions.`

Amaç net tanımlanmadığında model tipik olarak şu hataları üretir:

- yanlış soyutlamalar kurabilir,
- gereksiz karmaşıklık üretebilir,
- alanla uyuşmayan yapılar oluşturabilir.

Amaç tanımı, modelin tüm sonraki kararlarını filtreleyen bir referans çerçevesi işlevi görür: her soyutlama, her katman ve her bağımlılık bu amaca hizmet edip etmediğine göre değerlendirilebilir hale gelir.

Alan Modelinin Kurulması

Yapay zekâ için en kritik yapılardan biri alan (domain) modelidir. Bu nedenle şunlar açık biçimde tanımlanmalıdır:

- ana varlıklar,
- ilişkiler,
- yaşam döngüleri,
- değişmezler (invariant'lar).

Örneğin:

Entities:

- Session
- Participant
- Transcript
- Recording
- Summary

Rules:

- One session can have many participants
- Transcript belongs to exactly one session
- Recording exists only during an active session

Alan modeli net olmadığında sistemin farklı bölümleri farklı gerçeklikler üretmeye başlar: aynı varlık farklı modüllerde farklı ilişki ve kısıtlarla temsil edilir ve bu ayrışma zamanla veri tutarsızlıklarına dönüşür. Açık alan modeli, hem insan hem de model için ortak bir kavramsal sözlük oluşturur.

Sınır Tanımları

Yapay zekânın en çok ihtiyaç duyduğu yapılardan biri, açık biçimde tanımlanmış sınırlardır (boundary). Katmanlı bir sınır tanımı şu şekilde ifade edilebilir:

UI Layer

|

v

Facade Layer

|

v

Application Services

|

v

Domain Layer

|

v

Infrastructure Layer

Bu yapı, aşağıdaki gibi açık kurallarla desteklenmelidir:

UI cannot directly call API.

Only facades orchestrate state changes.

Bu tür kurallar, modelin kontrolsüz bağlaşım üretmesini belirgin biçimde azaltır. Sınır tanımı yapılmadığında tipik bozulmalar şunlardır:

- iş mantığı arayüz katmanına gömülür,
- yan etkiler sisteme dağılır,
- bağımlılık grafiği karmaşıklaşır.

Sınırlar, modelin “bu kod nereye yazılmalı” sorusunu her seferinde yeniden çözmesini engeller; karar, mimari düzeyde bir kez verilmiş olur.

Durum Geometrisinin Tanımlanması

Modern arayüz mimarisinde durum organizasyonu kritik öneme sahiptir. Bu nedenle modele durum sahipliği açık kurallarla verilmelidir:

Global state:

- auth
- session
- workspace context

Local state:

- modal visibility
- temporary form edits

Never store transient UI state globally.

Bu ayrım yapılmadığında model tipik olarak şu hataları üretir:

- her durumu küresel depoya taşır,
- eşitleme problemleri üretir,
- kontrolsüz yeniden çizim zincirleri oluşturur.

Durum geometrisi tanımı, özünde bir sahiplik haritasıdır: hangi bilginin nerede yaşadığı, kimin tarafından değiştirilebildiği ve hangi ömre sahip olduğu önceden belirlenir.

Veri Akış Yönlerinin Tanımlanması

Modele veri akışının yönü açık biçimde verilmelidir. Tek yönlü bir akış tanımı şu şekilde ifade edilebilir:

UI -> Action -> Effect -> API -> Reducer -> Store -> UI

Bu yaklaşım üç temel bozulmayı önlemeye yardımcı olur:

- çift yönlü ve izlenemez veri akışı,
- gizli yan etkiler,
- durum belirsizliği.

Akış yönü tanımlandığında, model bir değişikliğin etki zincirini deterministik biçimde izleyebilir: hangi eylemin hangi dönüşümü tetiklediği ve verinin hangi yoldan arayüze döndüğü önceden bellidir.

Hata Modelinin Tasarlanması

Modern sistemlerde yalnızca mutlu yol (happy path) senaryosunu tanımlamak yeterli değildir. Bu nedenle modele sistemin başarısızlık koşulları açık sorular olarak yöneltilmelidir:

What happens if:

- the realtime connection disconnects?
- an upload fails?
- retry exceeds its limit?
- a race condition occurs?

Bu soruların yanıtları, sistemin davranış geometrisini dayanıklı hale getirir. Hata modeli tanımlanmadığında model, başarısızlık durumlarını ya hiç ele almaz ya da her modülde farklı ve tutarsız biçimlerde ele alır. Merkezî bir hata modeli, başarısızlığın da tasarlanmış bir davranış olmasını sağlar.

Ölçek Varsayımlarının Belirlenmesi

Modelin doğru karmaşıklık seviyesini kurabilmesi için sistemin hedef ölçeği belirtilmelidir:

Expected scale:

- realtime communication
- 10k concurrent users
- distributed architecture
- offline capability

Bu bilgi verilmediğinde model iki yönde de hata yapabilir:

- gereksiz karmaşıklık üretebilir,
- yetersiz ölçeklenebilirlik kurabilir,
- yanlış soyutlama seviyesi seçebilir.

Ölçek varsayımı, mimari kararların maliyet-fayda dengesini belirleyen temel parametredir: bin kullanıcı bir sistem için doğru olan mimari, milyon kullanıcı bir sistem için yanlış olabilir ve tersi de geçerlidir.

Kod Organizasyonunun Tanımlanması

Modele proje yapısı ve organizasyon biçimi açıkça verilmelidir. Örneğin dizin düzeyinde:

```
apps/  
libs/  
shared/  
features/  
domain/  
infrastructure/
```

veya ilke düzeyinde:

Feature-first architecture

gibi yönlendirmeler sistem bütünlüğünü artırır. Kod organizasyonu tanımı, modelin ürettiği her yeni dosyanın öngörülebilir bir konuma yerleşmesini sağlar; böylece sistem büyüdükçe yapısal tutarlılık korunur.

Test Geometrisinin Tanımlanması

Modelin yalnızca kod değil, doğrulama yapısı da üretmesi gerekir. Bu nedenle test katmanları açık biçimde belirtilmelidir:

- **unit**
- **contract**
- **integration**
- **E2E**
- **concurrency**
- **drift guards**

Test geometrisi tanımlanmadığında model tipik olarak şu davranışları gösterir:

- yetersiz test üretir,
- yüzeysel anlık görüntü testlerine yönelir,
- davranış geometrisini doğrulayamaz.

Test katmanlarının önceden tanımlanması, doğrulamanın sonradan eklenen bir katman değil, mimarinin kurucu unsuru olmasını sağlar.

Mimari Prensiplerin Tanımlanması

Son olarak, modele sistemin temel prensipleri açık biçimde verilmelidir:

Principles:

- deterministic state transitions
- minimal side effects
- explicit boundaries
- modular design
- typed contracts
- low coupling
- high cohesion
- AI-readable structure

Bu prensipler, sistemin entropi üretme eğilimini baştan sınırlar. Prensip listesi, modelin karşılaştığı her belirsiz durumda başvurabileceği bir karar hiyerarşisi işlevi görür: iki geçerli çözüm arasında seçim yapılması gerektiğinde, prensiplere daha uygun olan tercih edilir.

Yaklaşımın Bütünsel Değerlendirmesi

Yukarıdaki on tanımlama adımı birlikte ele alındığında, yapay zekâya iyi mimari hazırlatmanın özü şu şekilde ifade edilebilir:

Yapay zekâya mimari tanımlamak, ona bir düşünme uzayı vermek anlamına gelir.

Bu farkı iki istem biçimi üzerinden karşılaştırmak öğreticidir. Kötü yaklaşım, tanımsız bir uzayda üretim talep eder:

Build me a collaboration app.

İyi yaklaşım ise üretimin gerçekleşeceği geometriyi önceden kurar:

Design a modular collaboration platform with deterministic state flow, explicit orchestration boundaries, typed contracts, streaming-safe concurrency handling, and scalable testing architecture.

Birinci istem, modeli sonsuz bir olasılık uzayında serbest bırakır; üretilen sistem, modelin o anki örüntü tercihlerinin rastgele bir birleşimi olur. İkinci istem ise olasılık uzayını mimari kısıtlarla daraltır; üretilen sistem, önceden tanımlanmış geometrinin bir gerçekleştirimi olur.

Bu karşılaştırma, yaklaşımın adımı da gerekçelendirir: geometri-önce yaklaşımda kod, tasarım sürecinin başlangıcı değil sonucudur. Sistemin amacı, sınırları, akışları ve prensipleri — yani geometrisi — önce tanımlanır; kod bu geometrinin içinde üretilir.

Sonuç

Yapay zekâ destekli mühendislikte üretim kalitesi, modelin kapasitesinden çok, modele sunulan yapısal çerçevenin kalitesiyle belirlenir. Bu çalışmada sistematikleştirilen on tanımlama adımı — sistem amacı, alan modeli, sınırlar, durum geometrisi, veri akış yönleri, hata modeli, ölçek varsayımları, kod organizasyonu, test geometrisi ve mimari prensipler — birlikte, modelin güvenilir biçimde akıl yürütebileceği bir uzay kurar.

Bu çerçevenin temel prensibi şu şekilde özetlenebilir:

Önce sistemin geometrisi tasarlanır, daha sonra kod üretilir. Kod sonuçtur; geometri ise sebeptir.

Bu bakış açısı, yapay zekâ çağında mühendislik rolünün dönüşümünü de işaret eder: mühendisin asli katkısı artık kodun kendisini yazmak değil, kodun içinde üretileceği geometriyi doğru kurmaktır. İyi kurulmuş bir geometri, hem insan hem de makine için üretimin doğruluğunu yapısal olarak güvence altına alır.